

Argus Security Analysis Report

Repository: https://github.com/arnav-sutraway/Vuln_test

Total Findings: 23

1. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: auth.py, line 22

Evidence: `f"WHERE username = '{username}' "`

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

2. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: auth.py, line 39

Evidence: + "" WHERE email = ""

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

3. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: database.py, line 11

Evidence: query = "SELECT * FROM users WHERE username = " + username + ""

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

4. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: database.py, line 24

Evidence: query = f"SELECT * FROM users WHERE id = {user_id}"

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

5. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: database.py, line 39

Evidence: `f"WHERE username = '{username}' "`

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

6. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: database.py, line 74

Evidence: query = f"SELECT * FROM products ORDER BY {sort_column}"

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

7. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: High

Confidence: 88.0%

Location: admin.py, line 55

Evidence: query = f"SELECT * FROM audit_logs WHERE action LIKE '%{search_term}%'"

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

8. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: services/users.py, line 10

Evidence: "WHERE email = " + email + " "

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

9. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: services/users.py, line 24

Evidence: query = f"DELETE FROM users WHERE id = {user_id}"

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

10. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: services/reports.py, line 10

Evidence: `f"WHERE type = '{report_type}' "`

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

11. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: services/reports.py, line 24

Evidence: query = f"SELECT * FROM {table_name}"

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

12. Reflected Cross-Site Scripting in Search Endpoint

Vulnerability Type: Cross-Site Scripting

Severity: High

Confidence: 98.0%

Location: app.py, line 14

Evidence: `html = f""" Search Results You searched for: {query} """`

Description: The search endpoint retrieves the query parameter 'q' directly from the request arguments and concatenates it into an HTML response string using an f-string without escaping or sanitization. The raw HTML response is then returned to the user with a text/html content type, enabling reflected Cross-Site Scripting.

Recommendation: Render the query using Flask's `render_template` with Jinja2 autoescaping enabled, or properly escape user input with `markupsafe.escape` before embedding it into HTML.

13. Reflected Cross-Site Scripting via Markup in Profile Endpoint

Vulnerability Type: Cross-Site Scripting

Severity: High

Confidence: 98.0%

Location: app.py, line 31

Evidence: `unsafe_content = Markup( f"Welcome, {username}" )`

Description: The profile route reads the 'username' query parameter and interpolates it into an HTML snippet that is explicitly wrapped in `markupsafe.Markup`. This explicitly marks untrusted user-controlled input as safe HTML, bypassing escaping mechanisms and causing reflected Cross-Site Scripting in the browser.

Recommendation: Do not wrap unescaped user input in `Markup`. Instead, use `Markup.format` or escape the input using `markupsafe.escape(username)` before embedding it into markup.

14. Stored Cross-Site Scripting in Admin Comments View

Vulnerability Type: Cross-Site Scripting

Severity: High

Confidence: 85.0%

Location: admin.py, line 42

Evidence: `return render_template( "admin.html", comments=comments )`

Description: User-submitted comments stored in the database are fetched and passed to the admin.html template. Because comments originate from untrusted users and are rendered in the administrative interface without proper contextual escaping or sanitization, this introduces a stored Cross-Site Scripting vulnerability allowing execution in the context of an administrator.

Recommendation: Ensure that the admin.html template relies on standard Jinja2 autoescaping and does not use the `|safe` filter or disable autoescaping when rendering user-submitted comments.

15. SQL Injection in Login Authentication

Vulnerability Type: SQL Injection

Severity: Critical

Confidence: 95.0%

Location: auth.py, line 20

Evidence: query = ("SELECT id, username, role FROM users " f"WHERE username = '{username}' " f"AND password_hash = '{password}'")

Description: The login function constructs an SQL query using Python f-string interpolation with the username and password parameters. If externally influenced, an attacker could manipulate the query logic to bypass authentication.

Recommendation: Use parameterized queries with placeholders: conn.execute('SELECT id, username, role FROM users WHERE username = ? AND password_hash = ?', (username, password)).

16. SQL Injection in Password Reset Query

Vulnerability Type: SQL Injection

Severity: High

Confidence: 95.0%

Location: auth.py, line 36

Evidence: query = ("UPDATE users SET password = " + new_password + " WHERE email = " + email + " ")

Description: The reset_password function directly concatenates new_password and email strings into an UPDATE SQL statement, allowing attackers to inject arbitrary SQL commands during password resets.

Recommendation: Rewrite the query using parameterized query placeholders: conn.execute('UPDATE users SET password = ? WHERE email = ?', (new_password, email)).

17. SQL Injection in User Authentication Query

Vulnerability Type: SQL Injection

Severity: Critical

Confidence: 95.0%

Location: database.py, line 37

Evidence: query = (f"SELECT * FROM users " f"WHERE username = '{username}' " f"AND password = '{password}'")

Description: The authenticate_user function interpolates username and password into a dynamic SQL query, enabling authentication bypass via SQL injection.

Recommendation: Use parameter binding: cursor = conn.execute('SELECT * FROM users WHERE username = ? AND password = ?', (username, password)).

18. SQL Injection in Product Search Query

Vulnerability Type: SQL Injection

Severity: High

Confidence: 90.0%

Location: database.py, line 54

Evidence: query = ("SELECT * FROM products " "WHERE name LIKE '%" + search_term + "%' AND category = '" + category + "'")

Description: The search_products function concatenates search_term and category into a dynamic SQL query, permitting arbitrary SQL injection through either input.

Recommendation: Use parameterized queries with wildcards passed as parameter values: conn.execute('SELECT * FROM products WHERE name LIKE ? AND category = ?', (f'%{search_term}%', category)).

19. SQL Injection in Comment Insertion

Vulnerability Type: SQL Injection

Severity: High

Confidence: 90.0%

Location: admin.py, line 18

Evidence: query = ("INSERT INTO comments (username, comment) VALUES ('" + username + "', '" + comment + "')")

Description: The save_comment function concatenates username and comment into an INSERT SQL query, allowing SQL injection during comment creation.

Recommendation: Use parameterized queries: conn.execute('INSERT INTO comments (username, comment) VALUES (?, ?)', (username, comment)).

20. SQL Injection via Untrusted Input in user_lookup

Vulnerability Type: SQL Injection

Severity: High

Confidence: 95.0%

Location: app.py, line 43

Evidence: user = get_user_by_name(username)

Description: User input from request.args.get('username') is passed directly into get_user_by_name, which concatenates it into an unparameterized SQL statement.

Recommendation: Ensure underlying database functions use parameterized queries or bind parameters before query execution.

21. SQL Injection via Untrusted Input in user_by_id

Vulnerability Type: SQL Injection

Severity: High

Confidence: 95.0%

Location: app.py, line 56

Evidence: `user = get_user_by_id(user_id)`

Description: User input from `request.args.get('id')` is passed to `get_user_by_id` without casting or validation, leading to SQL injection in the underlying query.

Recommendation: Validate or cast `user_id` to an integer, and ensure `get_user_by_id` uses parameterized SQL execution.

22. SQL Injection in User Lookup Query

Vulnerability Type: SQL Injection

Severity: High

Confidence: 90.0%

Location: services/users.py, line 8

Evidence: query = ("SELECT * FROM users " "WHERE email = '" + email + "' " "AND role = '" + role + "'")

Description: The find_users function constructs a dynamic query by concatenating email and role parameters, creating a SQL injection vulnerability.

Recommendation: Use parameterized queries: conn.execute('SELECT * FROM users WHERE email = ? AND role = ?', (email, role)).

23. SQL Injection in Report Generation Query

Vulnerability Type: SQL Injection

Severity: High

Confidence: 90.0%

Location: services/reports.py, line 8

Evidence: query = ("SELECT * FROM reports " f"WHERE type = '{report_type}' " f"AND owner_id = {owner_id}")

Description: The generate_report function embeds report_type and owner_id directly into an SQL string using f-string formatting without parameter binding.

Recommendation: Use parameterized queries: conn.execute('SELECT * FROM reports WHERE type = ? AND owner_id = ?', (report_type, owner_id)).